Softwareentwicklung mit Java 2

5. Generics

5. Generics

- 5.1 Motivation
- 5.2 Grundlagen
- 5.3 Constraints
- 5.4 Ko- und Kontravarianz und Wildcards
- 5.5 Type Erasure
- 5.6 Generische Typen und Arrays
- 5.7 Weiteres
- 5.8 Fallstudie List
- 5.9 Zusammenfassung

© H. Prähofer, 2026

Problemanalyse: Nicht-generische Liste

Arbeiten mit Daten von beliebigem Referenztyp

➔ Liste mit Elementtyp **Object**

➔ Keine statische Prüfung des beabsichtigten Elementtyps

Typ des Parameters **Object**

```
public class List {

  public void add(Object obj) {
    ...
  }

  public Object get(int i) {
    return ...;
  }
}
```

Typ des Rückgabewerts **Object**

Beispiel: Liste von Students

```
List students = new List();
```

```
students.add(new Student("Hans"));
students.add(new Student("Fritz"));
```

```
students.add(new Professor("Wirth"));
```

Okay, weil **Professor** ist **Object**

```
Student hans = (Student) students.get(0);
Student fritz = (Student) students.get(1);
```

```
Student x = (Student) students.get(2);
```

ClassCastException

© H. Prähofer, 2026

Problemanalyse: Methoden ohne Generizität

Methoden für beliebige Referenztypen

➔ Typ **Object** für Parameter und Rückgabewert

➔ Keine Typsicherheit durch Compiler

Typ des Rückgabewerts **Object**

```
public Object find(List elems, Object o) {
    for (Object e : elems) {
        if (e.equals(o)) return e;
    }
    return null;
}
```

Typ des Parameters **Object**

```
List persons = new List();
persons.add(new Person(...));
persons.add(new Person(...));
```

```
Person p = new Person(...);
Object objFound = find(persons, p);
```

```
Person personFound = (Person) find(persons, p);
```

Typecast mit potentiellen **ClassCastExceptions**

Ziele:

- **Statische Typsicherheit:** Typfehler werden bereits vom Compiler erkannt.
- **Typspezifische Zugriffe**: Explizite Typumwandlungen sind nicht erforderlich.
- **Wiederverwendbarkeit**: Klassen und Methoden können für unterschiedliche Typen verwendet werden.

Lösung:

- **Generische Typen** und **Methoden** mit **Typparametern**

© H. Prähofer, 2026

5. Generics

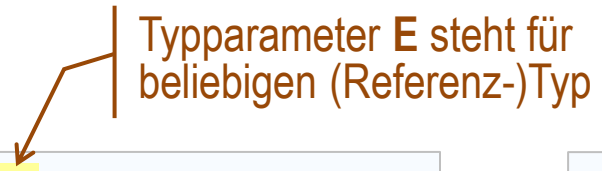
- 5.1 Motivation
- 5.2 Grundlagen
- 5.3 Constraints
- 5.4 Ko- und Kontravarianz und Wildcards
- 5.5 Type Erasure
- 5.6 Generische Typen und Arrays
- 5.7 Weiteres
- 5.8 Fallstudie List
- 5.9 Zusammenfassung

© H. Prähofer, 2026

Generische Typen [1/3]

Generische Typen: Klassen und Interfaces mit Typparameter

- Typparameter sind Variablen für Typen
- Generische Typen sind Schablonen für konkrete parametrisierte Typen




```
public interface List<E> {
  public void add(E elem);
  public E get(int i);
  …
}
```

```
public class LinkedList<E> implements List<E> {
  public void add(E elem) { … }
  public E get(int i) { … }
  …
}
```

Parametrisierte Typen

- Durch Konkretisierung der Typparameter entstehen konkrete parametrisierte Typen

```
List<String> stringList;
```

```
List<String> stringList = new LinkedList<String>();
```

© H. Prähofer, 2026

Generische Typen [2/3]

```
public interface List<E> {
  public void add(E elem);
  public E get(int i);
  …
}
```

```
public class LinkedList<E> implements List<E> {
  public void add(E elem) { … }
  public E get(int i) { … }
  …
}
```

Erzeugen von konkreten parametrisierten Typen

- durch "Zuweisung" von Typen zu Typparametern:

```
List<String> stringList;
```

E = String und
damit ist folgender List-Typ verbunden

```
public interface List<String> {
  public void add(String elem);
  public String get(int i);
  …
}
```

```
List<String> stringList = new LinkedList<String>();
```

E = String und
damit ist folgender LinkedList-Typ verbunden

```
public class LinkedList<String> implements List<String> {
  public void add(String elem) { … }
  public String get(int i) { … }
  …
}
```

Es werden aber keine solchen Klassen oder Interfaces erzeugt! Siehe Type Erasure

© H. Prähofer, 2026

Generische Typen [3/3]

```
List<String> stringList = new LinkedList<String>();
```



```
public class LinkedList<String> implements List<String> {
  public void add(String elem) { … }
  public String get(int i) { … }
  …
}
```

```
List<Person> personList = new LinkedList<Person>();
```



```
public class LinkedList<Person> implements List<Person> {
  public void add(Person elem) { … }
  public Person get(int i) { … }
  …
}
```

Typsicheres Arbeiten

- durch konkrete Typen für Typparameter bei Variablen, Methodenparametern und Rückgabewerten

```
stringList.add("Hello");
```

add mit Parameter vom Typ **String**

```
String s = stringList.get(0);
```

get mit Rückgabe von Typ **String**

```
stringList.add(new Person(...));
```



```
personList.add(new Person(...));
```

add mit Parameter vom Typ **Person**

```
Person p = personList.get(0);
```

get mit Rückgabe vom Typ **Person**

```
personList.add("Hello");
```



© H. Prähofer, 2026

Mehrere Typparameter

Beispiel: Map mit Schlüsseln und Werten

```
class Map<K, V> {
  ...
  void put(K key, V value) {...}
  V get(K key) {...}
}
```

Verwendung

```
Map<String, Person> map = new Map<String, Person>();
map.put("Meier", aPerson);
Person maier = map.get("Meier");
```

```
Map<Integer, String> map = new Map<Integer, String>();
map.put(17, "Pizza");
String s = map.get(17);
```

Instanziierung mit "Diamond"-Operator

```
Map<String, Person> map1 = new Map<>();
Map<Integer, String> map2 = new Map<>();
```

Diamond-Operator

Bei der Objekterzeugung können die Typargumente weggelassen werden, wenn sie aus dem Kontext abgeleitet werden können..

© H. Prähofer, 2026

Generischen Typen und Vererbung

Bei **Ableitung von einem generischen Typ** muss der **Typparameter** festgelegt werden durch

- einen konkreten Typ

Typparameter von List gleich **String**

```
class StringList extends List<String> {...}
```

- einen Typparameter der abgeleiteten Klasse

Typparameter von **List** gleich zu dem von **ExtList**

```
class ExtList<E> extends List<E> {...}
```

Typparameter von **List** ist **List<E>**
mit **E** gleich zu dem von **ListOfLists**

```
class ListOfLists<E> extends List<List<E>> {...}
```

Gilt analog für die Implementierung und Erweiterung von Interfaces!

Nicht erlaubt:

- Verwendung nicht deklarierter Typparameter

```
class SubClass extends BaseClass<E>
```

✗

```
class SubClass<E> extends BaseClass<U>
```

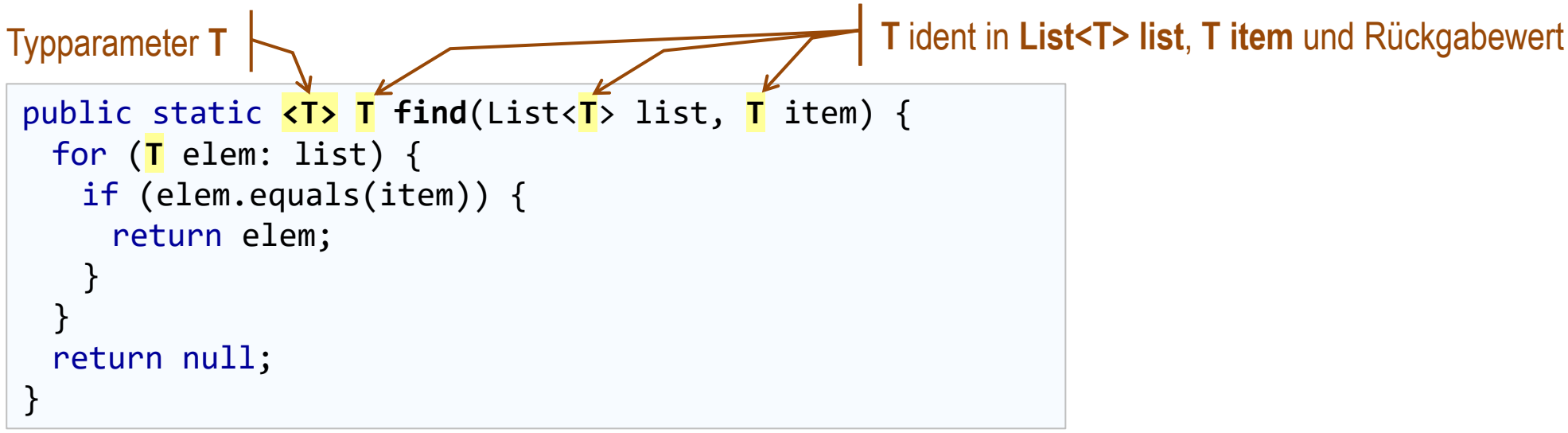
✗

© H. Prähofer, 2026

Generische Methoden

Methodendeklaration mit Typparameter

- **Typparameter** zum Bestimmen der Typen von **Parameter**, **Variablen** und **Rückgabewerten**




```
public static <T> T find(List<T> list, T item) {
  for (T elem: list) {
    if (elem.equals(item)) {
      return elem;
    }
  }
  return null;
}
```

- Ableitung des **konkreten Typs** aus den **aktuellen Methodenparametern**

```
List<String> keywords = …;
String found = find(keywords, "lambda");
```

Typ **T = String**

```
List<Person> persons = …;
Person found = find(persons, new Person(…));
```

Typ **T = Person**

© H. Prähofer, 2026

Lernziele überprüfen

- Was ist ein generischer Typ in Java und wie wird er deklariert?
- Wie entstehen parametrisierte Typen durch Angabe konkreter Typargumente?
- Wie können generische Methoden mit Typparametern definiert werden?
- Wie wird bei einem Methodenaufruf der konkrete Typ für einen Typparameter bestimmt?
- Wie werden Typparameter einer generischen Basisklasse bei der Vererbung festgelegt?
- Wozu dient der Diamond-Operator < > bei Objekterzeugung?

© H. Prähofer, 2026

5. Generics

- 5.1 Motivation
- 5.2 Grundlagen
- 5.3 Typconstraints
- 5.4 Ko- und Kontravarianz und Wildcards
- 5.5 Type Erasure
- 5.6 Generische Typen und Arrays
- 5.7 Weiteres
- 5.8 Fallstudie List
- 5.9 Zusammenfassung

© H. Prähofer, 2026

Typconstraints: Problemanalyse

Gesucht: Sortierte Liste für Elemente von Typ **E**

```
class SortedList<E> {
  E[] data = ...;
  ...
  void add(E elem) { ... }
}
```

Ohne Typconstraint sind für Objekte vom Typ **E** nur die Methoden von **Object** garantiert

Frage: Wie soll man Objekte zum Sortieren vergleichen?

Lösung:

- Typparamter **E** einschränken, dass nur Typen erlaubt sind, die **Comparable** implementieren
- Damit könnte man die Elemente von Typ **E** mit **compareTo** vergleichen

```
public interface Comparable<T> {
  int compareTo(T other);
}
```

< 0 ... this ist kleiner als other
== 0 ... this und other sind gleichrangig
> 0 ... this ist größer als other

© H. Prähofer, 2026

Typconstraints : Bounded Type Parameters

`T extends TypeBound`

TypeBound kann Interface oder Klasse sein

- Das Typargument für **T** muss ein Subtyp von ***TypeBound*** sein oder ***TypeBound*** selbst.
- Mit der Typgarantie sind Operationen von ***TypeBound*** garantiert

```
public class SortedList<E extends Comparable<E>> {
  E[] es = ...;
  ...
  public void add(E e) {
    int i = size - 1;
    while (i >= 0 && es[i].compareTo(e) > 0)  {
      es[i+1] = es[i];
      i--;
    }
    es[i+1] = e;
    size++;
  }
}
```

```
public interface Comparable<T> {
  int compareTo(T obj);
}
```

Achtung:
Verbesserte Version siehe Wildcards!

© H. Prähofer, 2026

Beispiel Comparable<T>: Verwendung

- Interface für Ordnungsrelation

```
public interface Comparable<T> {
  int compareTo(T obj);
}
```

- Implementierung durch Klasse **Person**

```
public class Person implements Comparable<Person> {
  private final String firstName;
  private final String lastName;
  private final int age;
  …
  public int compareTo(Person other) {
    int c = this.lastName.compareTo(other.lastName);
    if (c != 0) return c;
    c =  this.firstName.compareTo(other.firstName);
    if (c != 0) return c;
    return Integer.compare(this.age, other.age);
  }
}
```

- **SortedList** für **Person** verwenden

```
SortedList<Person> sortedPersons = new SortedList<>();
```

Person erfüllt Constraint **Person extends Comparable<Person>**

© H. Prähofer, 2026

Typconstraints bei Methoden

Typconstraints auch für Typparameter von Methoden

- Beispiel Methode **max**

```
public static <T extends Comparable<T>> T max(T x, T y) {
  if (x.compareTo(y) >= 0) return x;
  else return y;
}
```

Achtung:
Verbesserte Version
siehe Wildcards!

- **max** für **Person** anwenden

```
Person person1 = new Person("Hans", "Maier", 32);
Person person2 = new Person("Hans", "Maier", 45);

Person greater = max(person1, person2);
System.out.println(greater);
```

```
Hans Maier, 45
```

© H. Prähofer, 2026

Mehrere Typconstraints möglich

```
class SortedMap<K extends Comparable<K>, V extends Comparable<V> & Serializable> {
  ...
  void put(K key, V value) {...}
  V get(K key) {...}
}
```

&

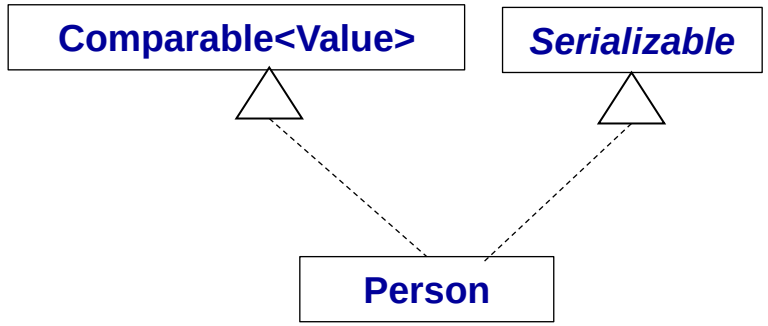
Verwendung

muss **Comparable<K>** implementieren

muss **Comparable<V>** und **Serializable** implementieren

```
SortedMap<String, Person> tab = new SortedMap<>();
...
tab.put("John", new Person(...));
```

```
class Person implements Comparable<Person>, Serializable {
  ...
  public int compareTo(Person p) {...}
}
```



Achtung: Bei mehreren Bounds muss eine Klasse, falls vorhanden, an erster Stelle stehen. Danach folgen Interfaces.

© H. Prähofer, 2026

Vererbung bei Typconstraints

Die an die generische Basisklasse übergebenen Typargumente müssen deren Type Bounds erfüllen

```
class BaseClass<TB extends A> {...}
class SubClass<TS extends B> extends BaseClass<TS> {...}
```

➔ **B** muss **gleich oder spezieller** als **A** sein

Beispiel:

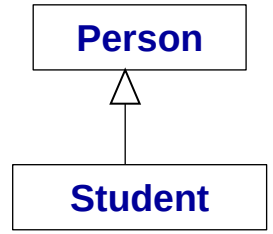
```
class PList<TB extends Person> { ... }

class SList<TS extends Student> extends PList<TS> {...}
```

TB extends Person in Basisklasse

TS extends Student in Unterklasse

erfüllt **TB extends Person** in Basisklasse



© H. Prähofer, 2026

Lernziele überprüfen

- Was ist ein Typconstraint in Java und wozu wird er verwendet?
- Wie wird ein Typconstraint für einem Typparameters angegeben?
- Wie werden mehrere Typconstraints kombiniert?
- Was bedeutet es, wenn Subklassen die Typconstraints der Basisklasse erfüllen müssen?

© H. Prähofer, 2026

5. Generics

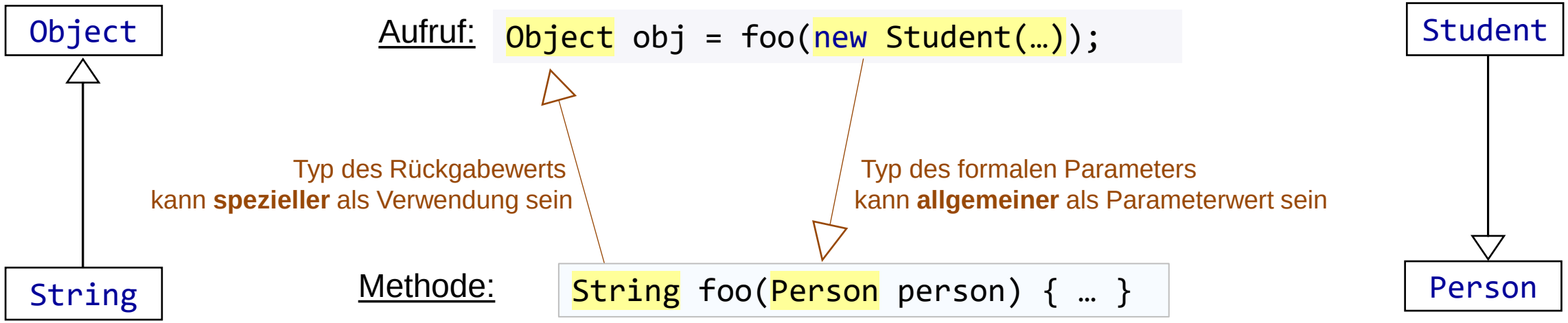
- 5.1 Motivation
- 5.2 Grundlagen
- 5.3 Constraints
- 5.4 Ko- und Kontravarianz und Wildcards
- 5.5 Type Erasure
- 5.6 Generische Typen und Arrays
- 5.7 Weiteres
- 5.8 Fallstudie List
- 5.9 Zusammenfassung

© H. Prähofer, 2026

Vorbemerkung: Kompatibilität bei Methodenaufrufen

Formaler Parameter und Rückgabewert im Vergleich zum Aufruf

- Der Typ des formalen Parameters darf ein Supertyp des Argumenttyps sein.
- Der Rückgabetyp der Methode darf ein Subtyp des Typs sein, in dem das Ergebnis verwendet wird.



Begründung: Zuweisungskompatibilität

- Objekt **Student** kompatibel mit Parameter **Person person**
- Rückgabewert **String** kompatibel mit Variable **Object obj**

© H. Prähofer, 2026

Motivation: Methode copyTo (Invariante Version)

```
public class List<E> {
  public int size() {...}
  public E get(int idx) {...}
  public void add(E x) {...}
}
```

- Methode **copyTo** soll alle Elemente von Liste **from** in Liste **to** kopieren

```
public static <T> void copyTo(List<T> from, List<T> to) {
  for (int i = 0; i < from.size(); i++) {
    T elem = from.get(i);
    to.add(elem);
  }
}
```

Invariante Version: Unflexible Lösung, weil bei beiden Listen der Elementtyp gleich sein muss!

```
List<Person> persons = new List<Person>();

List<Student> students = new List<Student>();
students.add(new Student(…));
```

```
copyTo(students, persons);
```

✗ **does not compile**

Ursachenanalyse:

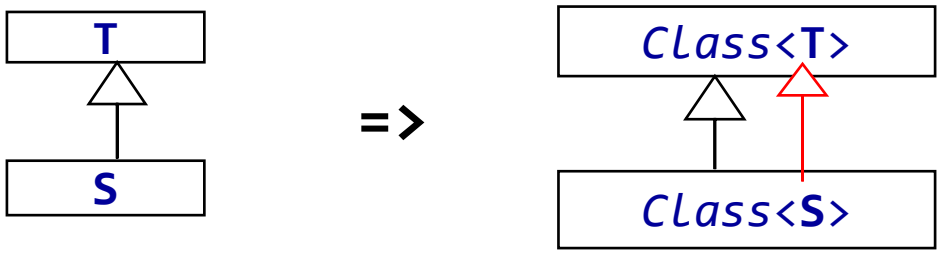
- Wenn **T = Person**, dann ist **from** von Typ **List<Person>**
- Parameterübergabe würde eine Zuweisung **List<Person> from = students** bedeuten
- Damit müsste **List<Student>** zuweisungskompatibel mit **List<Person>** sein
- Das wäre gegeben, wenn **List<Student>** ein Subtyp von **List<Person>** wäre, was aber nicht der Fall ist

© H. Prähofer, 2026

Kovarianz bei generischen Typen: Problemanalyse

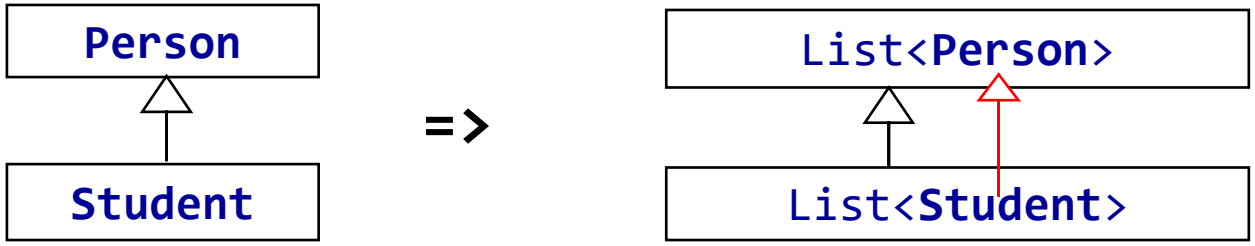
Kovarianz bei generischen Typen würde heißen

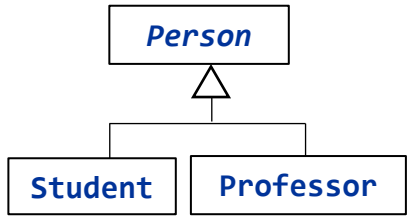
- Wenn **S** Subtyp von **T** => ***Class*<S>** Subtyp von ***Class*<T>**



In Java so nicht unterstützt!

Beispiel: Student Subtyp von **Person** => **List<Student>** Subtyp von **List<Person>**





Mit **Kovarianz** wäre folgende **kovariante Zuweisung** möglich:

```
List<Person> persons = new List<Student>();
```

✗ **nicht typsicher**

© H. Prähofer, 2026

Kovarianz bei generischen Typen: Problemanalyse

Kovariante Zuweisungen sind nicht typsicher

```
List<Student> students = new List<Student>();
students.add(new Student(...));
```

persons und **students** zeigen auf gleiche Liste

```
List<Person> persons = students;
```

✗ kovariante Zuweisung nicht erlaubt

persons würde Einfügen von **Professor** erlauben

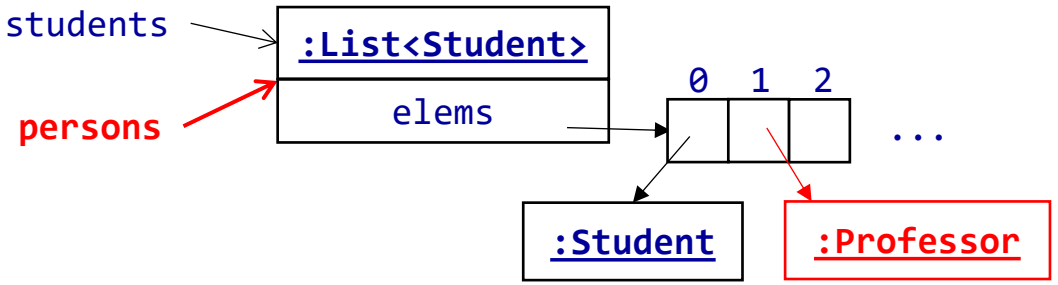
```
persons.add(new Professor(...));
```

students garantiert **Student** Wert ist aber **Professor**

```
Student s = students.get(1);
```

ClassCastException

```
public class List<E> {
  public int size() {...}
  public E get(int idx) {...}
  public void add(E x) {...}
}
```



➔ Kovariante Zuweisungen in dieser Weise in Java nicht erlaubt

© H. Prähofer, 2026

Upper-bounded Wildcard

Upper-bounded Wildcard erlaubt kovariante Zuweisungen

`<? extends T>`

<? extends T> steht für einen unbekannten Typ, der **T** selbst oder ein Subtyp von **T** ist.

Unbounded Wildcard:

`<?>` = `<? extends Object>`

- Variablendeklarationen mit upper-bounded Wildcard

```
List<? extends T> someTList
```

Damit erlaubt **someTList** alle **List**-Objekte mit Typparameter gleich **T** oder spezieller als **T**

- Kovariante Zuweisung erlaubt, weil **List<Student>** ist mit **List<? extends Person>** kompatibel

```
List<Student> students = new List<Student>();
students.add(new Student(..));
List<? extends Person> persons = students;
```

kovariante Zuweisung

Typinferenz:
? wird zu **Student**
Wildcard-Constraint wird zu **Student extends Person**
und dies ist erfüllt

© H. Prähofer, 2026

Upper-bounded Wildcard

Upper-bounded Wildcard erlaubt nur lesende Zugriffe

```
List<Student> students = new List<Student>();
students.add(new Student(..));
```

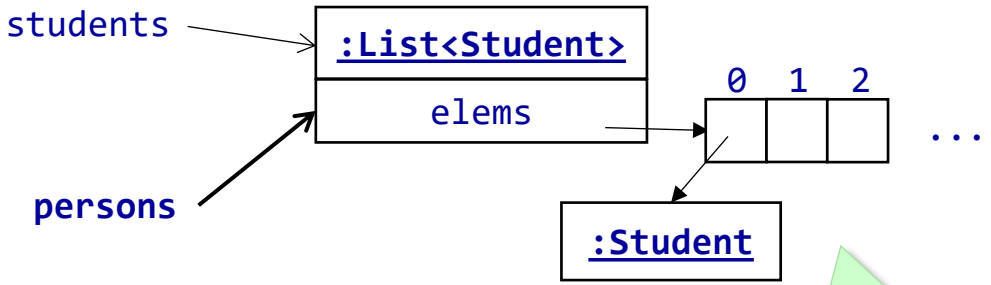
```
List<? extends Person> persons = students;
```

```
public class List<E> {
  public int size() {...}
  public E get(int idx) {...}
  public void add(E x) {...}
}
```

- Methoden mit **generischem Rückgabewert** sind **typsicher** und erlaubt

```
Person p = persons.get(0);
```

persons.get garantiert **Person**, **Student** kompatibel mit **Person**



- aber Methoden mit **generischem Parameter** werden vom **Compiler verboten**

```
persons.add(new Professor(...));
```

✗

extendsPerson.add würde auch **Professor** erlauben, ist aber nicht kompatibel mit **Student**

Kein **add** über Variable **persons** erlaubt!

© H. Prähofer, 2026

copyTo: Kovariante Version

copyTo: Lösung mit **Upper-Bounded Wildcard**

```
public static <T> void copyTo(List<? extends T> from, List<T> to) {
  for (int i = 0; i < from.size(); i++) {
    T elem = from.get(i);
    to.add(elem);
  }
}
```

from *kovariant*

Lesen von *kovarianter Liste* **from**

- Anwendung beim Kopieren von **students** zu **persons**

```
List<Student> students = new List<Student>();
```

```
List<Person> persons = new List<Person>();
```

```
copyTo(students, persons);
```

Typinferenz:

1) Ableitung von Typparameter **T** aus aktuellem Argument **persons** für zweiten Parameter **to**:
   - ➔ **persons** ist von Typ **List<Person>**
   - ➔ **T** = **Person**
   - ➔ **from** ist von Typ **List<? extends Person>**

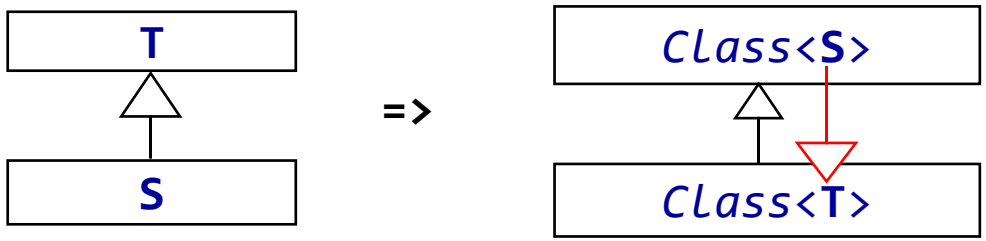
2) Überprüfen des aktuellen Parameters **students** für ersten Argument **from**:
   - ➔ **students** ist von Typ **List<Student>**
   - ➔ Damit wird Wildcard **?** zu **Student**
   - ➔ und Wildcard-Constraint **Student extends Person** ist erfüllt

© H. Prähofer, 2026

Kontravarianz bei generischen Typen: Problemanalyse

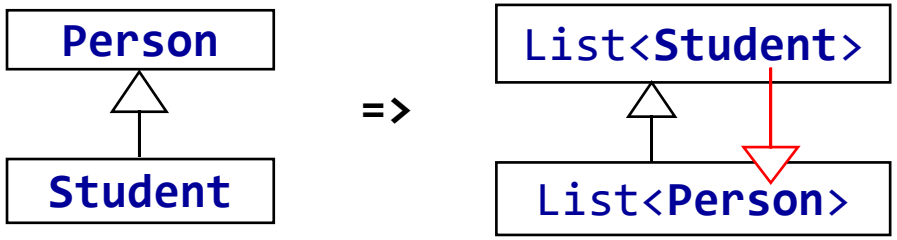
Kontravarianz bei generischen Typen würde heißen

Wenn **S** Subtyp von **T** => ***Class*<T>** Subtyp von ***Class*<S>**



In Java so nicht unterstützt!

Beispiel: Student Subtyp von **Person** => **List<Person>** Subtyp **von List<Student>**



Mit **Kontravarianz** wäre folgende **kontravariante Zuweisung** möglich:

```
List<Student> students = new List<Person>();
```

✗ **nicht typsicher**

© H. Prähofer, 2026

Kontravarianz bei generischen Typen: Problemanalyse

Kontravariante Zuweisungen sind nicht typsicher

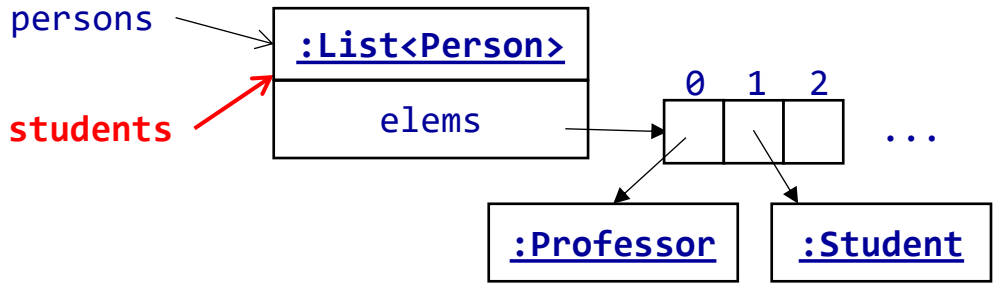
```
List<Person> persons = new List<Person>();
persons.add(new Professor(...));
```

```
public class List<E> {
  public int size() {...}
  public E get(int idx) {...}
  public void add(E x) {...}
}
```

persons und **students** zeigen auf gleiche Liste

```
List<Student> students = persons;
```

✗ konrtavariante Zuweisung nicht erlaubt



students.add erlaubt Einfügen von **Student** ➔ **okay**

```
students.add(new Student(...));
```

students.get garantiert **Student** ist aber **Professor** ➔ **nicht okay**

```
Student s = students.get(0);
```

ClassCastException

➔ **Kontravariante Zuweisungen in dieser Weise in Java nicht erlaubt**

© H. Prähofer, 2026

Lower-bounded Wildcard

Lower-bounded Wildcard erlaubt kontravariante Zuweisungen

`<? super T>`

<? super T> steht für einen unbekannten Typ, der **T** selbst oder ein Supertyp von **T** ist.

- **Variablendeklaration** mit **lower-bounded Wilcard**

```
List<? super T> superOfTList
```

Damit erlaubt **superOfTList** alle **List**-Objekte mit Typparameter gleich **T** oder allgemeiner als **T**

- Kontravariante Zuweisung erlaubt, weil **List<Person>** ist mit **List<? Super Student>** kompatibel

```
List<Person> persons = new List<Person>();
persons.add(new Professor(...));
List<? super Student> students = persons;
```

kontravariante Zuweisung

Typinferenz:
? wird zu **Person**
Wildcard-Constraint wird zu **Person super Student**
und dieser Constraint ist erfüllt

© H. Prähofer, 2026

Lower-bounded Wildcard

Lower-bounded Wildcards erlauben das typsichere Einfügen

Beim Lesen ist lediglich Object als Rückgabetyp garantiert

```
public class List<E> {
  public int size() {...}
  public E get(int idx) {...}
  public void add(E x) {...}
}
```

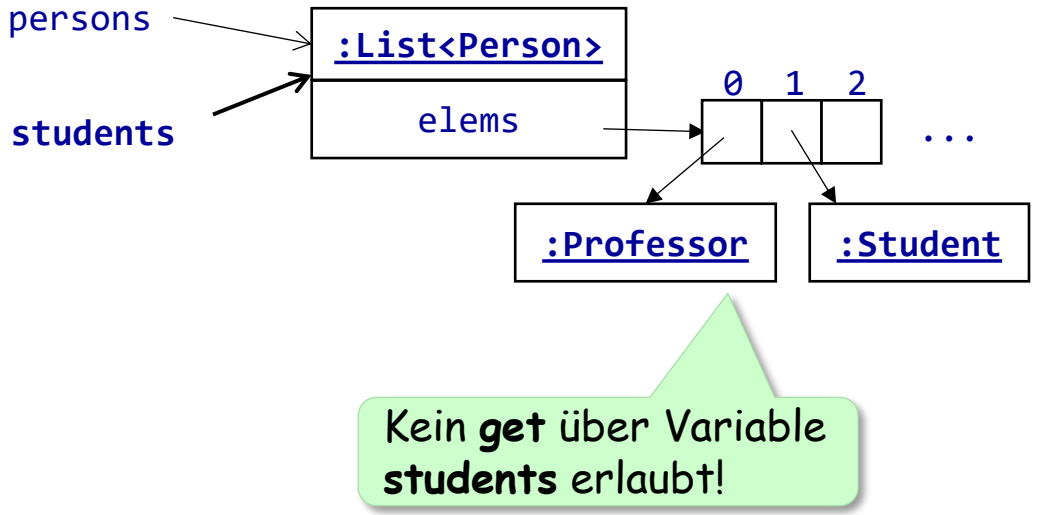
```
List<Person> persons = new List<Person>();
persons.add(new Professor(...));
```

```
List<? super Student> students = persons;
```

- Methoden mit generischen Parameter sind typsicher und erlaubt

```
students.add(new Student(...));
```

students.add erlaubt **Student**, ist kompatibel mit **Person**



- Lesen mit spezifischem Typ nicht möglich

```
Student s = students.get(0);
```

✗

students.get verboten, weil **Student** nicht garantiert werden kann!

© H. Prähofer, 2026

copyTo: Kontravariante Version

copyTo: Lösung mit **Lower-Bounded Wildcard**

```
public static <T> void copyTo(List<T> from, List<? super T> to) {
  for (int i = 0; i < from.size(); i++) {
    T elem = from.get(i);
    to.add(elem);
  }
}
```

to *kontravariant*

Schreiben auf *kontravariante Liste* **to**

- Anwendung beim Kopieren von **students** zu **persons**

```
List<Student> students = new List<Student>();
```

```
List<Person> persons = new List<Person>();
```

```
copyTo(students, persons);
```

Typinferenz:

1) Ableitung von Typparameter **T** aus aktuellem Argument **students** für ersten Parameter **from**:
   - ➔ **students** ist von Typ **List<Student>**
   - ➔ **T** = **Student**
   - ➔ **to** ist von Typ **List<? super Student>**

2) Überprüfen des aktuellen Argument **persons** für zweiten Argument **to**:
   - ➔ **persons** ist von Typ **List<Person>**
   - ➔ Damit wird Wildcard **?** zu **Person**
   - ➔ und Wildcard-Constraint **Person super Person** ist erfüllt

© H. Prähofer, 2026

Beispiel: Typebound bei Comparable

Typebound <T extends Comparable<T>> ohne Varianz problematisch

```
public static <T extends Comparable<T>> T max(T x, T y) {
  if (x.compareTo(y) >= 0) return x;
  else return y;
}
```

```
public interface Comparable<T> {
  int compareTo(T obj);
}
```

Problemanalyse

- Implementierung von **Comparable** bei **Person**

```
public class Person implements Comparable<Person> { … }
```

- und eine Subklasse **Student**, die Implementierung von **Comparable<Person>** erbt

```
public class Student extends Person { … }
```

- das erlaubt aber noch NICHT die Verwendung von **max** für **Student**

```
Student greater = max(new Student(…), new Student(…));
```

✗ **Typebound nicht erfüllt**

Typinferenz:
T = Student und Typebound **<T extends Comparable<T>>**
➔ Typebound ist **Comparable<Student>>** aber **Student** erbt nur **Comparable<Person>**

© H. Prähofer, 2026

Beispiel: Typebound bei Comparable

Lösung: Lower-Bounded Wildcard Comparable<? super T>

```
public static <T extends Comparable<? super T>> T max(T x, T y) {
  if (x.compareTo(y) >= 0) return x;
  else return y;
}
```

```
public interface Comparable<T> {
  int compareTo(T obj);
}
```

Begründung

- Implementierung von **Comparable** bei **Person**

```
public class Person implements Comparable<Person> { … }
```

- und eine Subklasse **Student**, die Implementierung von **Comparable<Person>** erbt

```
public class Student extends Person { … }
```

- mit Lower-Bounded Wildcard ist **SortedList<Student>** typ-korrekt

```
Student greater = max(new Student(…), new Student(…));
```

Typinferenz:
T = Student und Typebound **<T extends Comparable<? super T>>**
➔ **Student** erbt **Comparable<Person>** ➔ Typebound **Comparable<Person super Student>>** ist erfüllt

© H. Prähofer, 2026

Beispiel: Typebound bei Comparable bei SortedSet

Lower-Bounded Wildcard bei Comparable

```
public class SortedList<T extends Comparable<? super T>> {
  …
}
```

```
public interface Comparable<T> {
  int compareTo(T obj);
}
```

- Mit **Person** implementiert **Comparable<Person>** erbt

```
public class Person implements Comparable<Person> { … }
```

- und **Student** erbt **Comparable<Person>**

```
public class Student extends Person { … }
```

- Erlaubt SortedSet<Student>

```
SortedList<Student> sortedStudents = new SortedList<>();
```

Typinferenz:
Typebound **<T extends Comparable<? super T>>**
➔ **T = Student** ➔ Typebound ist **<Student extends Comparable<? super Student>>**
➔ **Student** erbt **Comparable<Person>** ➔ Typebound **Student extends Comparable<Person super Student>** ist erfüllt

© H. Prähofer, 2026

Lernziele überprüfen

- Warum sind kovariante und kontravariante Zuweisungen bei Generics problematisch?
- Was versteht man unter einem upper-bounded Wildcard, was unter einem lower-bounded Wildcard und wie werden diese angegeben?
- Welche Zugriffe sind bei einem upper-bounded Wildcard erlaubt, welche nicht?
- Welche Zugriffe sind bei einem lower-bounded Wildcard erlaubt, welche nicht?

© H. Prähofer, 2026

5. Generics

- 5.1 Motivation
- 5.2 Grundlagen
- 5.3 Constraints
- 5.4 Ko- und Kontravarianz und Wildcards
- 5.5 Type Erasure
- 5.6 Generische Typen und Arrays
- 5.7 Weiteres
- 5.8 Fallstudie List
- 5.9 Zusammenfassung

© H. Prähofer, 2026

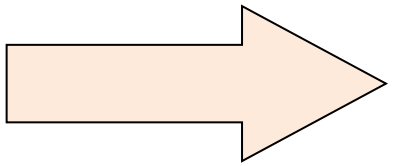
Type Erasure und Rohtypen

- Bei Type Erasure werden Typparameter durch ihre oberen Bounds ersetzt
  - Das ist für Typparameter ohne Typconstraint der Typ **Object**
  - Und mit der Typconstraint der Typconstraint, z.B. mit <T extends Comparable<T> der Typ Comparable

Generischer Typ

```
class List<E> {
  E[] data;
  List(int size) {...}
  void add(E x) {...}
  E get(int i) {...}
}
```

wird übersetzt zu



Rohtyp

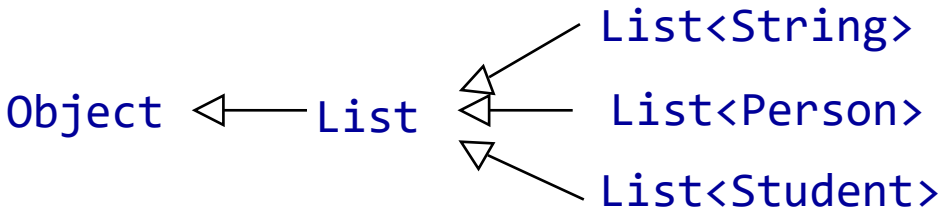
```
class List {
  Object[] data;
  List(int size) {...}
  void add(Object x) {...}
  Object get(int i) {...}
}
```

- Generische Typargumente werden zur Compilezeit geprüft
- und stehen aber zur Laufzeit nicht zur Verfügung

© H. Prähofer, 2026

Konsequenzen von Type Erasure [1/4]

- **Kompatibilitätsbeziehungen:** Alle generischen Typen sind mit Rohtyp kompatibel.



- Kovariante Zuweisungen auf Rohtyp sind damit erlaubt!

```
List<Student> studentList = new List<Student>();
List objList = studentList;
```

kovariante Zuweisung auf Rohtyp erlaubt ☹

- Damit wird aber die generische Typprüfung umgangen
  ➔ man hat beim Rohtyp genau die Probleme, die man mit Wildcards vermeiden wollte

```
objList.add(new Professor(...));
Student s = studentList.get(0);
```

ClassCastException!

© H. Prähofer, 2026

Konsequenzen von Type Erasure [2/4]

Laufzeitprüfungen funktionieren bei Generics nicht

```
Object obj = new List<String>();
```

- Prüfen mit generischem Typ nicht möglich

```
if (obj instanceof List<String>) ...
```

✗

Compilefehler: VM kann nicht prüfen, ob **obj** vom Typ **List<String>** ist

- Prüfen mit **<?>** erlaubt

```
if (obj instanceof List<?>) ...
```

Bevorzugt gegenüber Rohtyp

- Prüfen auf Rohtyp erlaubt

```
if (obj instanceof List) ...
```

Rohtyp

- Typecast erlaubt, aber mit Warning

```
List<String> list = (List<String>) obj;
```

Warnung: VM garantiert, dass **obj** vom Typ **List** ist, kann aber nicht garantieren, dass **obj** vom Typ **List<String>** ist

© H. Prähofer, 2026

Konsequenzen von Type Erasure [3/4]

Basisdatentypen können nicht für Typparameter verwendet werden

- Basisdatentypen sind nicht kompatibel mit **Object** des Rohtyps.

```
List<int> ints = new List<int>();
```

✗

Compilefehler: **int** kann nicht für Typparameter verwendet werden

- Man muss geboxten Referenztypen verwenden, z.B. mit **Integer**

```
List<Integer> ints = new List<Integer>();
ints.add(43);
```

geboxte Werte weniger effizient!

© H. Prähofer, 2026

Konsequenzen von Type Erasure [4/4]

Exception-Klassen dürfen nicht generisch sein

```
public class GenericException<T> extends Exception {
    ...
}
```

✗

Compilefehler: Kein Typparameter bei Exception-Klassen erlaubt!

Begründung

- catch-Klauseln führen zur Laufzeit eine Typprüfung des Exception-Wertes durch
- Generische Typen können nicht unterschieden werden

```
try {
    ...
} catch (GenericException<String> se) {
    ...
} catch (GenericException<Object> oe) {
    ...
}
```

✗

VM kann die Exception-Klassen nicht unterscheiden

© H. Prähofer, 2026

Lernziele überprüfen

- Was bedeutet „Type Erasure“ bei Generics?
- Was versteht man unter einem Rohtyp in Java?
- Auf Rohtypen sind kovariante Zuweisungen erlaubt. Welche Konsequenzen ergeben sich daraus?
- Warum ist instanceof List<String> nicht erlaubt, während instanceof List<?> zulässig ist?
- Warum dürfen Basistypen (z. B. int) nicht als Typparameter verwendet werden?
- Warum sind generische Exception-Klassen in Java nicht erlaubt

© H. Prähofer, 2026

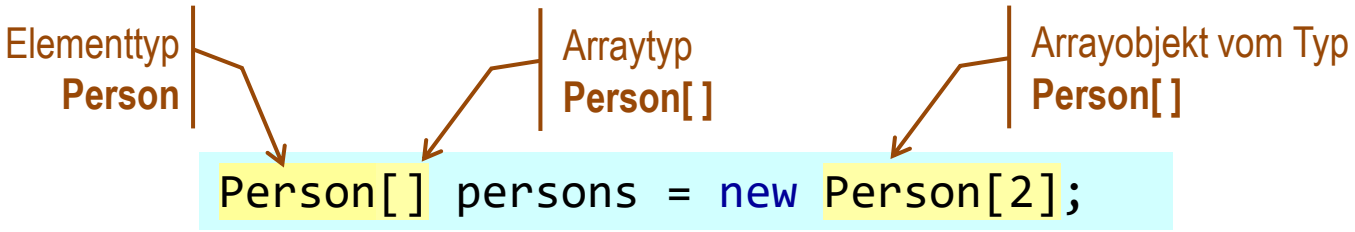
5. Generics

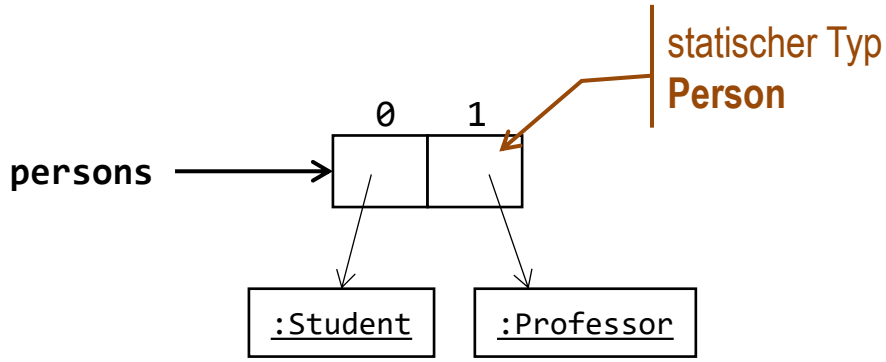
- 5.1 Motivation
- 5.2 Grundlagen
- 5.3 Constraints
- 5.4 Ko- und Kontravarianz und Wildcards
- 5.5 Type Erasure
- 5.6 Generische Typen und Arrays
- 5.7 Weiteres
- 5.8 Fallstudie List
- 5.9 Zusammenfassung

© H. Prähofer, 2026

Arrays als "generisches" Konzept

- Array-Operator **[ ]** erzeugt aus beliebigem Elementtyp **E** einen typisierten Arraytyp **E[ ]**





- Positionen im Array **E[ ]** haben statischen Typ **E**
  ➔ erlauben Elemente von Subtypen von **E**

```
persons[0] = new Student(...);
persons[1] = new Professor(...);
```

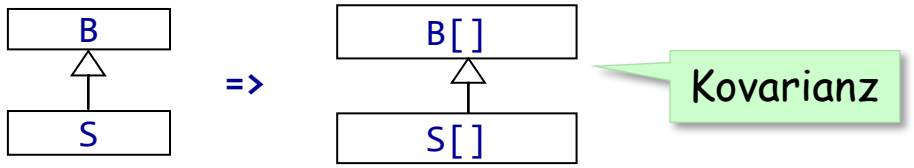
Student und **Professor** kompatibel mit statischem Typ **Person**

© H. Prähofer, 2026

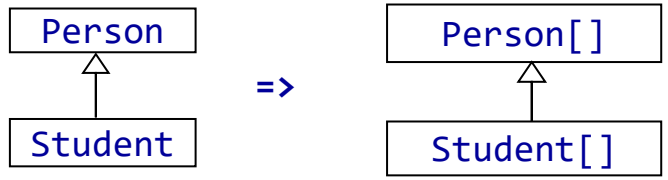
Arrays als "generisches" Konzept: Kovarianz [1/2]

Arrays sind kovariant

- Array **S[ ]** mit speziellerem Elementtyp **S** kompatibel mit Array **B[ ]** mit allgemeinerem Elementtyp **B**



Beispiel:

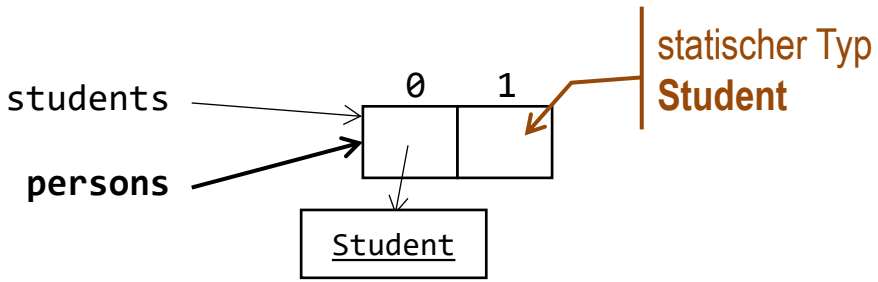


- **Kovariante Zuweisung bei Arrays erlaubt**

```
Student[] students = new Student[2];
students[0] = new Student(...);
```

```
Person[] persons = students;
```

Kovariante Zuweisung erlaubt !



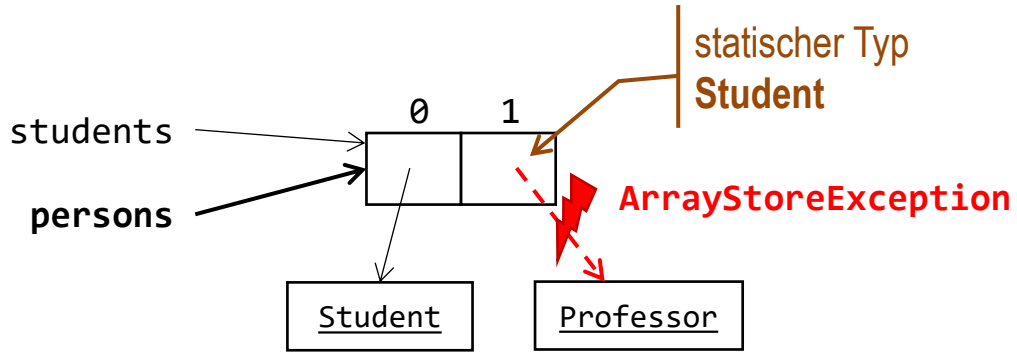
© H. Prähofer, 2026

Arrays als "generisches" Konzept: Kovarianz [2/2]

```
Student[] students = new Student[2];
students[0] = new Student(...);
```

```
Person[] persons = students;
```

Kovariante Zuweisung erlaubt !



Konsequenzen

- Lesender Zugriff **typsicher**

```
Person p = persons[0];
```

statischer Typ **Person**

dynamischer Typ **Student**

Person[] garantiert **Person** an Stelle 0,
Wert ist von Typ **Student** ➔ OK

- Schreibende Zugriffe können zur Laufzeit **ArrayStoreException** auslösen

```
persons[1] = new Professor(...);
```

ArrayStoreException

Person[] persons erlaubt jede **Person** an Stelle 1, also auch einen **Professor**, Array ist aber vom Typ **Student[]**

➔ **Prüfung zur Laufzeit** und **ArrayStoreException**

© H. Prähofer, 2026

Arrays von generischen Typen

Erzeugen von Arrays mit Typparametern nicht möglich!

```
class ArrayList<E> {
  private E[] ts = new E[16];
  ...
}
```

✗

Ebenso Arrays von konkretisierten generischen Typen nicht möglich!

```
List<String>[] stringListArray =
                new List<String>[10];
```

✗

Erlaubt ist Array eines konkreten Typs und Typecast zu generischem Array:

- entweder Array mit allgemeinstem Typ **Object**

```
class ArrayList<E> {
  private E[] ts = (E[]) new Object[CAP];
  ...
}
```

- oder Array von Typebound

```
class SortedList<E extends Comparable<? super E>> {
  private E[] ss = (E[]) new Comparable[16];
  ...
}
```

Rohtyp des Typebounds!

Erlaubt ist Array vom Rohtyp und Typecast

```
List<String>[] stringListArray =
      (List<String>[]) new List[10];
```

Rohtyp!

© H. Prähofer, 2026

Lernziele überprüfen

- Warum sind Arrays in Java „kovariant“ und welche Probleme entstehen daraus?
- Warum ist das Erzeugen von Arrays mit Typparametern (new T[]) nicht erlaubt?
- Welche Unterschiede bestehen zwischen Arrays und Generics in Bezug auf Typsicherheit?
- Wie kann man innerhalb einer generischen Klasse ein Array zur Speicherung von Elementen des Typparameters erzeugen? Welche Einschränkungen sind dabei zu beachten?

© H. Prähofer, 2026

5. Generics

- 5.1 Motivation
- 5.2 Grundlagen
- 5.3 Constraints
- 5.4 Ko- und Kontravarianz und Wildcards
- 5.5 Type Erasure
- 5.6 Generische Typen und Arrays
- 5.7 Weiteres
- 5.8 Fallstudie List
- 5.9 Zusammenfassung

© H. Prähofer, 2026

Typinferenz [1/2]

Teil des Compilers

Typinferenz leitet die Typen aus dem Kontext ab

- Keyword **var:** Typ einer Variablen wird vom Initialwert abgeleitet

```
var x = new List<String>();
```

Typ von **x** ist **List<String>**

- Methoden mit Parameter: es erfolgt die Ableitung der Typen aus aktuellen Argumenten

```
public static <T> List<T> of(T...ts) {
    List<T> list = new List<T>();
    for (T t: ts) list.add(t);
    return list;
}
```

```
var strings = List.of("Hallo", "World");
```

Ableitung von **List<String>**

```
var objects = List.of("Hallo", new Point(1, 2), new Student( … ));
```

Ableitung eines gemeinsamen Obertyps der Argumenttypen

© H. Prähofer, 2026

Typinferenz [2/2]

Typinferenz leitet die Typen aus dem Kontext ab

- Generische Methoden ohne Argumente:
  Ableitung derTypargumente aus dem Aufrufkontext
  - z.B. aus Typ der Variablen bei Zuweisung

```
List<Student> emptyStudentList = List.emptyList();
```

List<Student>

```
public class List<E> implements Iterable<E> {
  public static <T> List<T> emptyList() {
    return new List<T>();
  }
  …
}
```

  - Wenn Kontext nicht ausreichend, dann **List<Object>**

```
var list = List.emptyList();
```

List<Object>

  - oder durch explizite Angabe eines Typs für den Typparameter

```
var emptyStudentList = List.<Student>emptyList();
```

List<Student>

© H. Prähofer, 2026

Objekterzeugung

- Direkte Objekterzeugung mit einem Typparameter (new T()) ist nicht erlaubt

```
new T();
```
✗

```
public class Box<T> {
    private T value = new T();

    public void init() {
        value = new T();
    }
}
```
✗

✗

© H. Prähofer, 2026

Objekterzeugung

- Objekterzeugung mit Typparameter **nicht** möglich

```
new T();
```
✗

```
public class Box<T> {
    T value;
    public void init() {
        value = new T();
    }
}
```
✗

- Alternative: Mit Erzeuger-Objekt (**Supplier**)

```
package java.util.function;

public interface Supplier<T> {
  T get();
}
```

```
public class Box<T> {
    T value;
    public void init(Supplier<T> creator) {
        value = creator.get();
    }
}
```

```
box.init(new Supplier<String>() {
    @Override
    public String get() {
        return "A";
    }
});
```

besser mit Lambdas ➔ siehe Kapitel Lambda Expressions

© H. Prähofer, 2026

Lernziele überprüfen

- Was versteht man unter Typinferenz und aus welchen Informationen kann der Compiler Typargumente ableiten?
- Welche Rolle spielt das Schlüsselwort **var** bei Typinferenz?
- Wie werden bei generischen Methoden die Typargumente aus den übergebenen Argumenten abgeleitet?
- Wie wird bei Methoden ohne Parameter der Typ abgeleitet (z. B. Collections.emptyList())?
- Warum ist die direkte Objekterzeugung mit Typparametern (new T()) nicht möglich?

© H. Prähofer, 2026

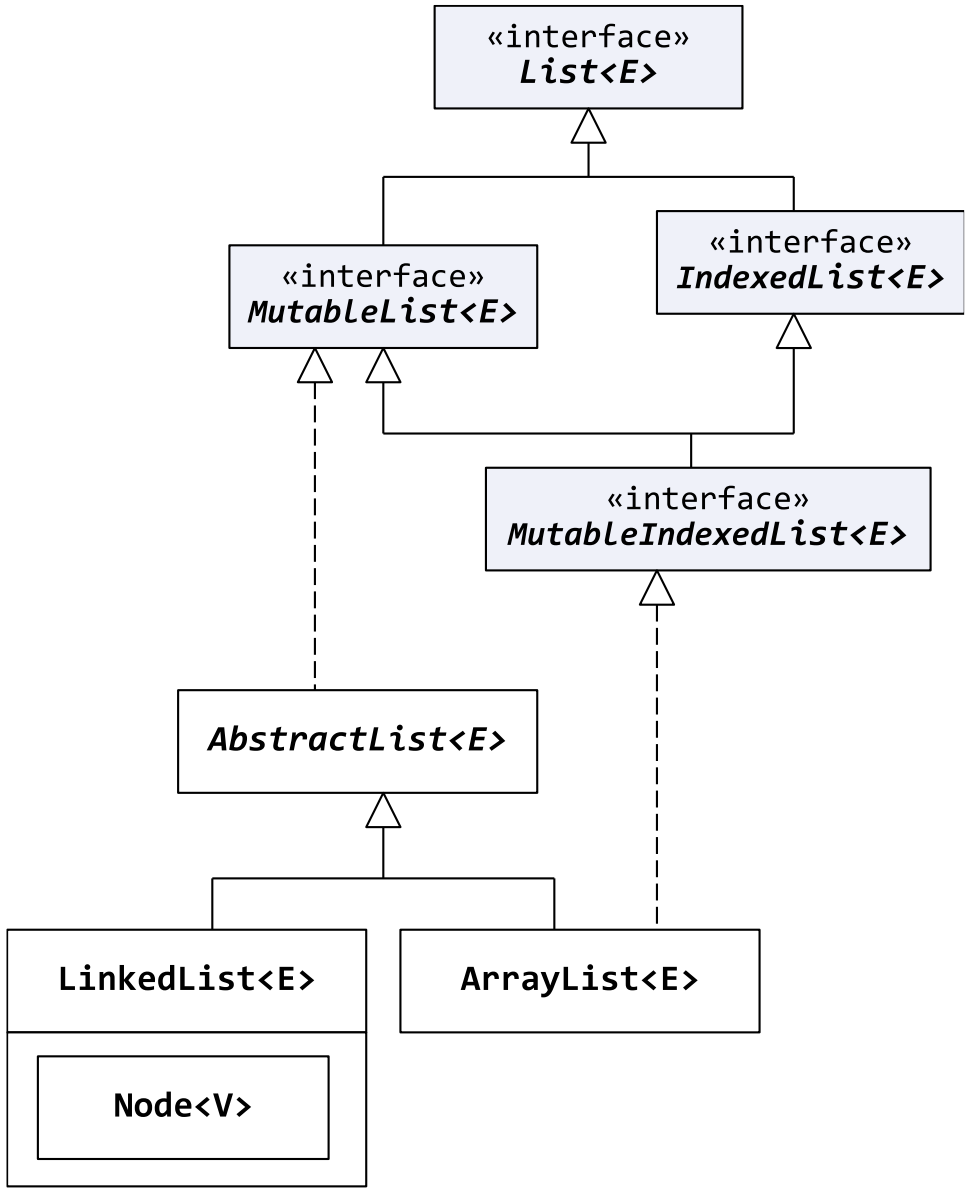
5. Generics

- 5.1 Motivation
- 5.2 Grundlagen
- 5.3 Constraints
- 5.4 Ko- und Kontravarianz und Wildcards
- 5.5 Type Erasure
- 5.6 Generische Typen und Arrays
- 5.7 Weiteres
- 5.8 Fallstudie List
- 5.9 Zusammenfassung

© H. Prähofer, 2026

Fallstudie List: Generische Interfaces und Klassen

Klassen List generisch



Decorators hier ausgelassen

© H. Prähofer, 2026

Fallstudie List: Generische Interfaces

- Interfaces **List<E>**, **IndexedList<E>**, **MutableList<E>** und **MutableIndexedList<E>**

```
public interface List<E> extends Iterable<E> {
  …
  public int size();
  default boolean contains(Object o) { … };
}
```

```
public interface IndexedList<E> extends List<E> {
  …
  E get(int idx);
  int indexOf(Object o);
  default boolean contains(Object o) { return indexOf(o) >= 0; }
}
```

```
public interface MutableList<E> extends List<E> {
  …
  void clear();
  void add(E elem);
  default void addAll(E... elems) {…}
  default void addAll(List<? extends E> list) {…}
  boolean remove(Object elem);
}
```

```
public interface MutableIndexedList<E> extends IndexedList<E> {
  …
  void add(int idx, E elem);
  E remove(int idx);
}
```

```
public interface Iterable<T> {
  Iterator<T> iterator();
  …
}
```

© H. Prähofer, 2026

Fallstudie List: Generische Klassen

- Klassen **AbstractList<T>** , **ArrayList<E>** und **LinkedList<T>**

```
public abstract class AbstractList<E> implements MutableList<E> {
  private int size = 0;

  protected AbstractList() { }

  @Override
  public int size() { return size; }
  …
}
```

```
public final class LinkedList<E> extends AbstractList<E> {

  private static class Node<V> {…}

  private Node<E> head;
  private Node<E> tail;
  …
}
```

```
public final class ArrayList<E> extends AbstractList<E> implements MutableIndexedList<E> {
  private E[] elems;
  public ArrayList(E...initial) {
    elems = (E[])new Object[INITIAL_CAPACITY];
    addAll(initial);
  }
  …
}
```

Erzeugen von **Object[]** aber Typecast zu **E[]**!

➔ siehe **Generische Typen und Arrays**

© H. Prähofer, 2026

Fallstudie List: Typparameter bei Statische Member-Klassen

- Statische Members benötigen eigenen Typparameter

```
public final class LinkedList<E> extends AbstractList<E> {

  private static class Node<V> {
    private final V elem;
    private Node<V> next;
    private Node<V> prev;

    private Node(V elem) {
      this.elem = elem;
    }
  }

  private Node<E> head;
  private Node<E> tail;
  …
}
```

Generische statische innere Klasse **Node** mit Typparameter **V**

Verwendung von **Node** mit Typ gleich dem Elementtyp **E** der **LinkedList<E>**

```
public interface List<E> extends Iterable<E> {
  …
  static final class ReadonlyListAdapter<T> implements List<T> { … }

  default List<E> asReadonlyList() {
    return new ReadonlyListAdapter<E>(this);
  }
}
```

© H. Prähofer, 2026

Fallstudie List: Typparameter bei Member-Klassen

- Member-Klassen verwenden Typparameter der äußeren Klasse

```
public final class LinkedList<E> extends AbstractList<E> {

  private class LinkedListIterator implements Iterator<E> {

    private Node<E> next = head;

    @Override
    public boolean hasNext() {
      return next != null;
    }

    @Override
    public E next() {
      if (!hasNext()) { throw new NoSuchElementException(); }
      E element = next.elem;
      next = next.next;
      return element;
    }
  }
  …
}
```

Innere Klasse **LinkedListIterator** ohne eigenen Typparameter

Innere Klasse **LinkedListIterator** verwendet Typparameter **E** der äußeren Klasse

© H. Prähofer, 2026

Fallstudie List: Typparameter bei Anonymen Klassen

- Anonyme Klassen können Typparameter von äußeren Klasse oder Methode verwenden

```
public class Lists {

  static <E> Iterator<E> iteratorOf(E[] array, int size) {

    return new Iterator<E>() {
      private int nextIndex;

      @Override
      public boolean hasNext() {
        return nextIndex < size;
      }

      @Override
      public E next() {
        if (!hasNext()) throw new NoSuchElementException();
        return array[nextIndex++];
      }
    };

  }
  …
}
```

Anonyme Klasse verwendet Typparamter **E** von Methode **iteratorOf**

© H. Prähofer, 2026

Fallstudie List: Adapters generisch

- Erzeugen von Listen aus Arrays und Objekten in Factory-Methode mit anonymen Klassen

```
public interface List<E> extends Iterable<E> {
  …
  static <E> IndexedList<E> of(E[] array) {
    return new IndexedList<E>() {

      @Override
      public int size() { return array.length; }

      @Override
      public E get(int idx) {
        if (idx < 0 || idx >= array.length) {
          throw new IndexOutOfBoundsException(idx);
        }
        return array[idx];
      }

      @Override
      public int indexOf(Object o) {
        for (int i = 0; i < array.length; i++) {
          if (Objects.equals(array[i], o)) return i;
        }
        return -1;
      }
       …
  }
```

```
…
  static <E> IndexedList<E> of(E single) {
    return new IndexedList<E>() {

      @Override
      public int size() {
        return 1;
      }

      @Override
      public E get(int idx) {
        if (idx == 0) return single;
        throw new IndexOutOfBoundsException(idx);
      }

      @Override
      public int indexOf(Object o) {
        return Objects.equals(single, o) ? 0 : -1;
      }
       …
  }
```

© H. Prähofer, 2026

Fallstudie List: Generische Zugriffe

Interface List<E>

- Zugriffe auf Elemente mit generischem Rückgabewert **T** sind typsicher

```
public interface Iterable<E> {
  public Iterator<E> iterator();
  …
}
```

```
public interface List<E> extends Iterable<E> {
  …
  public int size();
  default boolean contains(Object o) { … };
}
```

```
public interface IndexedList<E> extends List<E> {
  …
  E get(int idx);
  int indexOf(Object o);
  default boolean contains(Object o) {
    return indexOf(o) >= 0;
  }
}
```

```
List<Student> students = …;
```

```
Iterator< Student > iterator = students.iterator();
while (iterator.hasNext()) {
  Student p = iterator.next();
  …
}
```

```
for (Student p: students) { … }
```

```
IndexedList<Student> students = …;
```

```
Student first = students.get(0);
```

© H. Prähofer, 2026

Fallstudie List: Generische Zugriffe

Interface MutableList<E>

- **add** typsicher

```
public interface MutableList<E> extends List<E> {
  void add(E elem);
  …
}
```

```
MutableList<Student> mutStudents = new ArrayList<>();

mutStudents.add(new Student("Anna"));
```

- **addAll** mit kovariantem Wildcard **List<? extends E>**
  - ☐ erlaubt auch Listen mit Subtypen

```
public interface MutableList<E> extends List<E> {
  …
  default void addAll(List<? extends E> elems) {
    for (E t: elems) add(t);
  }
}
```

Z.B. Anfügen von **List<Student>** zu **MutableList<Person>**

```
MutableList<Person> mutPersons = new ArrayList<>();

List<Student> students = List.of( … );

mutPersons.addAll(students);
```

List<Student> kompatibel mit **List<? extends Person>**

© H. Prähofer, 2026

Fallstudie List: Methoden remove, contains nicht-generisch

- Die Methoden **contains** und **remove** mit Parameter von Typ **Object**

```
public interface List<E> extends Iterable<E> {
  …
  default boolean contains(Object o) { … };
}
```

Object ?!

```
public interface MutableList<E> extends List<E> {
  …
  boolean remove(Object elem);
}
```

Object ?!

- Frage: Warum werden die Methoden nicht generisch mit Parametertyp **E** definiert?

```
public interface List<E> extends Iterable<E> {
  …
  default boolean contains(E o) { … };
}
```

```
public interface MutableList<E> extends List<E> {
  …
  boolean remove(E elem);
}
```

© H. Prähofer, 2026

Fallstudie List: Problemanalyse remove

Interface MutableList<T>

- **Annahme: remove** mit Parameter von generischem Typ **E**

```
public interface MutableList<E> extends List<E> {
  …
  void remove(E elem);
  …
}
```

Beispiel Methode **transfer**: Lesen von **to**, kopieren in **to**, Löschen in **from**

✗ does not compile

```
public static <T> void transfer(MutableList<? extends T> from, MutableList<T> to) {
  List<? extends T> fromCopy = from.asReadonlyList();
  for (T elem: fromCopy) {
    to.add(elem);
    from.remove(elem);
  }
}
```

Methode **remove** kann für Liste mit kovariantenTyp **List<? extends T>** nicht verwendet werden!

Ursachenanalyse:

- Kovarianter Parameter **List<? extends T> from**
- Methode **remove** mit generischem Parameter kann bei kovariantem Parameter nicht verwendet werden

© H. Prähofer, 2026

Fallstudie List: remove nicht generisch

Lösung

- **remove** nicht generisch, sondern mit Parameter von Typ **Object**

```
public interface MutableList<T> extends List<T> {
  …
  void remove(Object elem);
  …
}
```

- **remove** nun bei kovariantem **MutableList<? extends Person> from** erlaubt

```
public static <T> void transfer(MutableList<? extends T> from, MutableList<T> to) {
  List<? extends T> fromCopy = from.asReadonlyList();
  for (T elem: fromCopy) {
    to.add(elem);
    from.remove(elem);
  }
}
```

- **remove** ist typsicher, weil es keine neuen (potentiell inkompatiblen) Elemente einfügt

```
MutableList<Person> mutPersons = new ArrayList<>( … );
MutableList<Student> mutStudents = new ArrayList<>( … );

transfer(mutStudents, mutPersons);
```

Gilt auch für **contains**, **indexOf** etc.

© H. Prähofer, 2026

Fallstudie List: removeAll mit kovariantem Wildcard List<?>

Interface MutableList<T>

- **removeAll** mit kovariantem *unbounded* Wildcard **List<?>**
  - erlaubt Listen mit beliebigem Elementtyp

```
public interface MutableList<T> extends List<T> {
  …
  default void removeAll(List<?> elems) {
    for (Object o: elems) remove(o);
  }
  …
}
```

Beachte:
<?> bedeutet **<? extends Object>**

```
MutableList<Student> mutStudents = new ArrayList<>();
mutStudents.add(…);

List<Person> persons = List.of(…);

mutStudents.removeAll(persons);
```

© H. Prähofer, 2026

5. Generics

- 5.1 Motivation
- 5.2 Grundlagen
- 5.3 Constraints
- 5.4 Ko- und Kontravarianz und Wildcards
- 5.5 Type Erasure
- 5.6 Generische Typen und Arrays
- 5.7 Weiteres
- 5.8 Fallstudie List
- 5.9 Zusammenfassung

© H. Prähofer, 2026

Zusammenfassung

- Generische Typen und Methoden verwenden Typparameter für ein Arbeiten mit unterschiedlichen Typen
  - Typparameter sind Variablen/Platzhalter für Typen
  - Werden durch konkrete Typen belegt
- Generizität dient der statischen Typprüfung zur Compilezeit
  - Generics ermöglichen typsicheres Arbeiten ohne explizite Typumwandlungen
- Generizität ist zu Vererbung und Typhierarchien orthogonal; dies erfordert:
  - Typconstraints schränken die zulässigen Typargumente ein.
  - Wildcards ermöglichen flexible Beziehungen zwischen parametrisierten Typen.
- Wildcards
  - Wildcards mit Upper Bounds ermöglichen kovariante Verwendung generischer Typen.
  - Wildcards mit Lower Bounds ermöglichen kontravariante Verwendung generischer Typen.
- Type Erasure in Java:
  - Generische Typen und Methoden werden auf eine gemeinsame Laufzeitimplementierung abgebildet
  - Typparameter werden durch ihren oberen Bound ersetzt (bei unbeschränkten Typparametern durch **Object**)
  - Konkrete Typargumente stehen zur Laufzeit für Typprüfungen nicht zur Verfügung

© H. Prähofer, 2026